

ITM SKILLS UNIVERSITY

School of FutureTech

DATABASE MANAGEMENT SYSTEMS

PROJECT REPORT

ESTATEHUB

Real Estate Property Listing & Enquiry Management System

- DBMS Case Study
- PostgreSQL + SQL
- Property Listings • Owners • Agents • Locations • Buyers • Enquiries
- Author: Sanika Kangane

Academic Year 2026–27

Table of Contents

- 1. Introduction
- 2. Problem Statement
- 3. Project Objectives
- 4. Scope and Requirements
- 5. Technology Stack
- 6. Database Architecture
- 7. ER Diagram and Relationships
- 8. Relational Schema
- 9. Normalization
- 10. Data Dictionary
- 11. PostgreSQL Implementation
- 12. SQL Queries Implemented
- 13. AvailableListings View
- 14. Testing and Validation
- 15. Integrity and Data Quality
- 16. Project Outcomes
- 17. Limitations and Future Enhancements
- 18. Conclusion
- Appendix A – Complete SQL Reference

1. Introduction

Real-estate management involves several connected categories of information. A property belongs to an owner, is located in a city, is handled by an agent, and may receive enquiries from multiple buyers. If these records are stored without a proper relational structure, information becomes repetitive and difficult to maintain.

EstateHub addresses this challenge by organizing the information into six related PostgreSQL tables. Primary keys uniquely identify records, foreign keys connect related records, and SQL queries provide the operations required for property search, statistics and enquiry tracking.

The design is intentionally database-centric. It can serve as the backend data model for a larger real-estate application while remaining a complete academic DBMS case study on its own.

2. Problem Statement

EstateHub requires a database for a property listing platform that stores property details, owner information, agent assignments, locations, buyers and buyer enquiries together. The database should allow agents to filter available listings by city and price range and efficiently follow up on buyer enquiries.

The design must also reduce unnecessary duplication. In particular, city and pincode information should not be repeated on every property record. A separate Location table is therefore used and linked to Property through LocationID.

3. Project Objectives

- Design an ER model for Property, Owner, Agent, Location, Buyer and BuyerPropertyEnquiry.
- Clearly identify primary keys and foreign keys.
- Convert the ER model into a normalized relational schema.
- Store city and pincode once in the Location table instead of repeating them on Property rows.
- Create PostgreSQL tables with appropriate NOT NULL, UNIQUE, CHECK, PRIMARY KEY and FOREIGN KEY constraints.
- Insert realistic sample records for owners, agents, locations, properties, buyers and enquiries.
- Filter available properties by city and a specified price range using JOIN and BETWEEN.
- Generate statistics using COUNT, AVG, GROUP BY and HAVING.
- Track buyer enquiries and follow-up status.
- Create an AvailableListings view joining property, owner, location and agent information.

4. Scope and Requirements

4.1 Functional Requirements

- Create and maintain property listings.
- Store property type, price and availability status.
- Maintain owner and agent contact information.
- Maintain reusable location records containing city and pincode.
- Register buyers.
- Record buyer enquiries against properties.
- Store enquiry date, message, follow-up status and follow-up date.
- Search available properties by city and price range.
- Generate property statistics by city, status and agent.
- Provide a consolidated AvailableListings view.

4.2 Non-Functional Requirements

- Consistency through primary and foreign key constraints.
- Reduced redundancy through normalization.
- Maintainability through clearly separated entities.
- Extensibility for future application modules.
- Reliable monetary storage using DECIMAL.
- Query efficiency through appropriate joins and indexes.

5. Technology Stack

- Database Management System: PostgreSQL
- Query Language: SQL
- Database Administration / Testing: pgAdmin and PostgreSQL Terminal
- Project SQL File: EstateHub.sql
- Project Type: DBMS Case Study – Real Estate Property Listing & Enquiry Management System

6. Database Architecture

EstateHub contains six main tables. Owner, Agent, Location and Buyer store relatively independent master data. Property acts as the central listing entity. BuyerPropertyEnquiry records the interaction between a buyer and a property.

- agent – stores real-estate agent details.
- buyer – stores buyer information.
- owner – stores property owner details.
- location – stores city and pincode information.
- property – stores property type, price, status and relationships to owner, location and agent.
- buyerpropertyenquiry – stores buyer enquiries and follow-up information.

7. ER Diagram and Relationships

The following is the single ER diagram used for the EstateHub database. It shows the six entities, their key attributes, relationship names and 1:M cardinalities.

EstateHub

ER Diagram — Real Estate Property Listing & Enquiry Management System

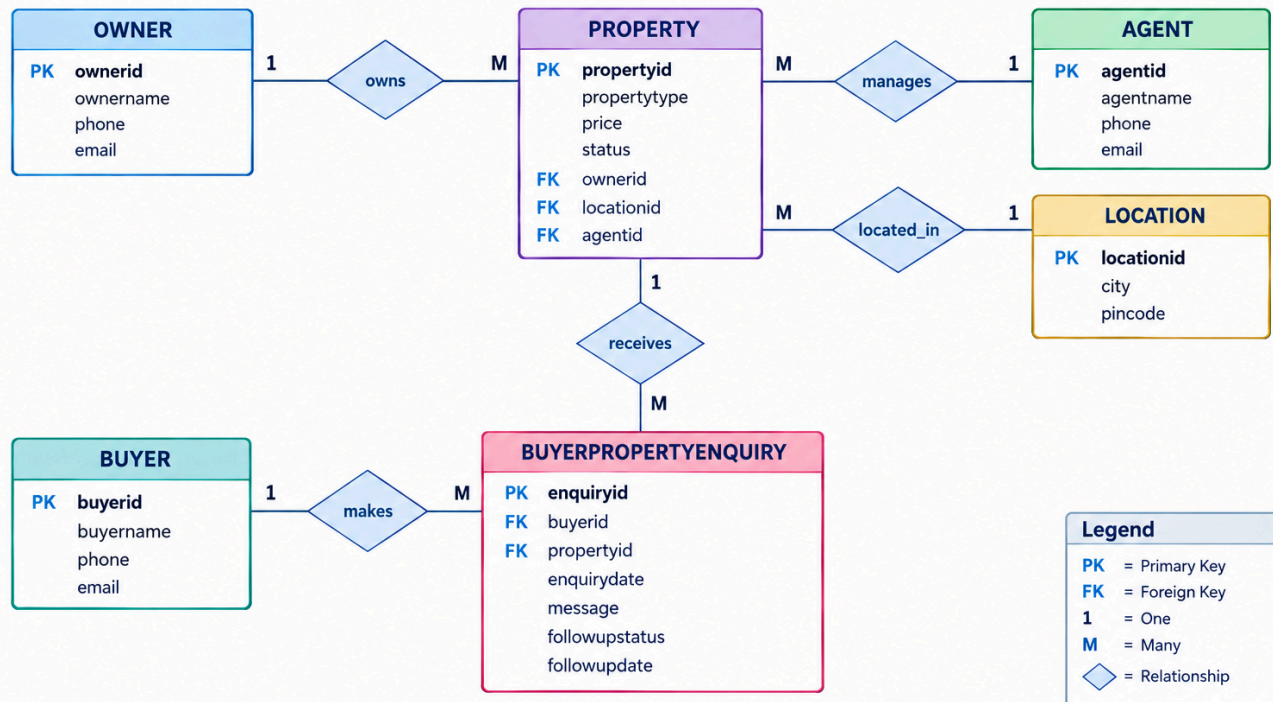


Figure 7.1 – EstateHub Entity–Relationship Diagram

7.1 Relationship Analysis

- Owner → Property (1:M): one owner can own multiple properties; each property references one owner.
- Agent → Property (1:M): one agent can manage multiple properties; each property is assigned to one agent.
- Location → Property (1:M): one location can contain multiple properties; each property references one location.
- Buyer → BuyerPropertyEnquiry (1:M): one buyer can create multiple enquiries.
- Property → BuyerPropertyEnquiry (1:M): one property can receive multiple enquiries.
- Buyer ↔ Property is effectively M:N through BuyerPropertyEnquiry. The enquiry table resolves this many-to-many interaction and stores additional attributes of the interaction.

7.2 Key Notation

- PK = Primary Key
- FK = Foreign Key
- 1 = One
- M = Many
- Relationship diamonds represent business relationships between entities.

8. Relational Schema

The relational schema follows the supplied ER model and uses the table names represented in the project README and ER diagram.

- OWNER(ownerid PK, ownername, phone, email)
- AGENT(agentid PK, agentname, phone, email)
- LOCATION(locationid PK, city, pincode)
- PROPERTY(propertyid PK, propertytype, price, status, ownerid FK, locationid FK, agentid FK)
- BUYER(buyerid PK, buyername, phone, email)
- BUYERPROPERTYENQUIRY(enquiryid PK, buyerid FK, propertyid FK, enquirydate, message, followupstatus, followupdate)

8.1 Key Design Decisions

- Each table has a simple identifier as its primary key.
- Property contains foreign keys to Owner, Location and Agent.
- BuyerPropertyEnquiry contains foreign keys to Buyer and Property.
- Location is separated from Property to avoid repeating city and pincode values.
- The enquiry table stores attributes that belong to the buyer-property interaction rather than to Buyer or Property itself.
- Property price is stored using a fixed-precision numeric type appropriate for monetary values.

9. Normalization

9.1 First Normal Form (1NF)

All attributes are atomic. A row represents one owner, one agent, one location, one property, one buyer or one enquiry. Repeating groups are not stored inside individual fields.

9.2 Second Normal Form (2NF)

The main relations use single-column primary keys, so non-key attributes do not depend on only part of a composite key. Separating Location from Property also eliminates repeated location details.

9.3 Third Normal Form (3NF)

Non-key attributes describe the key of their own relation rather than another non-key attribute. Owner details depend on ownerid, agent details on agentid, and location details on locationid. Property stores the IDs needed to reference those records instead of duplicating their descriptive attributes.

9.4 Representative Functional Dependencies

- ownerid → ownername, phone, email
- agentid → agentname, phone, email
- locationid → city, pincode
- propertyid → propertytype, price, status, ownerid, locationid, agentid
- buyerid → buyername, phone, email
- enquiryid → buyerid, propertyid, enquirydate, message, followupstatus, followupdate

10. Data Dictionary

Table	Field	Type	Key / Constraint	Purpose
owner	ownerid	INTEGER	PK	Unique owner ID
owner	ownername	VARCHAR	NOT NULL	Owner name
owner	phone	VARCHAR	NOT NULL	Owner contact
owner	email	VARCHAR	NOT NULL / UNIQUE	Owner email
agent	agentid	INTEGER	PK	Unique agent ID
agent	agentname	VARCHAR	NOT NULL	Agent name
agent	phone	VARCHAR	NOT NULL	Agent contact
agent	email	VARCHAR	NOT NULL / UNIQUE	Agent email
location	locationid	INTEGER	PK	Unique location ID
location	city	VARCHAR	NOT NULL	City
location	pincode	VARCHAR	NOT NULL	Postal pincode
property	propertyid	INTEGER	PK	Unique property ID
property	propertytype	VARCHAR	NOT NULL	Property category
property	price	NUMERIC	NOT NULL	Property price
property	status	VARCHAR	NOT NULL	Availability status
property	ownerid	INTEGER	FK / NOT NULL	References owner
property	locationid	INTEGER	FK / NOT NULL	References location
property	agentid	INTEGER	FK / NOT NULL	References agent
buyer	buyerid	INTEGER	PK	Unique buyer ID
buyer	buyername	VARCHAR	NOT NULL	Buyer name
buyer	phone	VARCHAR	NOT NULL	Buyer contact
buyer	email	VARCHAR	NOT NULL / UNIQUE	Buyer email
buyerpropertyenquiry	enquiryid	INTEGER	PK	Unique enquiry ID
buyerpropertyenquiry	buyerid	INTEGER	FK / NOT NULL	References buyer
buyerpropertyenquiry	propertyid	INTEGER	FK / NOT NULL	References property
buyerpropertyenquiry	enquirydate	DATE	NOT NULL	Enquiry date
buyerpropertyenquiry	message	TEXT/VARCHAR	NOT NULL	Buyer message
buyerpropertyenquiry	followupstatus	VARCHAR	NOT NULL	Follow-up state
buyerpropertyenquiry	followupdate	DATE	NULL	Follow-up date

11. PostgreSQL Implementation

11.1 Database Creation

EstateHub is implemented for PostgreSQL. Unlike some database systems, PostgreSQL does not use a USE database_name statement. The database is created first and the remaining statements are executed after connecting to estatehub_db.

```
CREATE DATABASE estatehub_db;
```

- In pgAdmin: create the database, connect to estatehub_db, open Query Tool, and execute the remaining SQL.
- In PostgreSQL Terminal (psql): create the database, then connect using \c estatehub_db.

11.2 Table Creation

```
CREATE TABLE owner (
    ownerid INT PRIMARY KEY,
    ownername VARCHAR(100) NOT NULL,
    phone VARCHAR(15) NOT NULL,
    email VARCHAR(120) NOT NULL UNIQUE
);

CREATE TABLE agent (
    agentid INT PRIMARY KEY,
    agentname VARCHAR(100) NOT NULL,
    phone VARCHAR(15) NOT NULL,
    email VARCHAR(120) NOT NULL UNIQUE
);

CREATE TABLE location (
    locationid INT PRIMARY KEY,
    city VARCHAR(80) NOT NULL,
    pincode VARCHAR(10) NOT NULL
);

CREATE TABLE buyer (
    buyerid INT PRIMARY KEY,
    buyername VARCHAR(100) NOT NULL,
    phone VARCHAR(15) NOT NULL,
    email VARCHAR(120) NOT NULL UNIQUE
);

CREATE TABLE property (
    propertyid INT PRIMARY KEY,
    propertytype VARCHAR(40) NOT NULL,
    price NUMERIC(14,2) NOT NULL CHECK (price > 0),
    status VARCHAR(20) NOT NULL
    CHECK (status IN ('Available','Sold','Reserved')),
    ownerid INT NOT NULL REFERENCES owner(ownerid),
    locationid INT NOT NULL REFERENCES location(locationid),
    agentid INT NOT NULL REFERENCES agent(agentid)
);

CREATE TABLE buyerpropertyenquiry (
    enquiryid INT PRIMARY KEY,
    buyerid INT NOT NULL REFERENCES buyer(buyerid),
    propertyid INT NOT NULL REFERENCES property(propertyid),
    enquirydate DATE NOT NULL,
    message VARCHAR(500) NOT NULL,
    followupstatus VARCHAR(20) NOT NULL
    CHECK (followupstatus IN ('Pending','Contacted','Closed')),
    followupdate DATE
);
```

11.3 Sample Data

The following sample records provide realistic test data for demonstrating the queries in this report.

```

INSERT INTO owner VALUES
(1,'Rajesh Mehta','9876500011','rajesh.mehta@example.com'),
(2,'Neha Shah','9876500012','neha.shah@example.com'),
(3,'Arjun Kapoor','9876500013','arjun.kapoor@example.com'),
(4,'Priya Nair','9876500014','priya.nair@example.com');

INSERT INTO agent VALUES
(101,'Amit Joshi','9810001001','amit.joshi@example.com'),
(102,'Sneha Patil','9810001002','sneha.patil@example.com'),
(103,'Rohan Desai','9810001003','rohan.desai@example.com');

INSERT INTO location VALUES
(201,'Mumbai','400001'),
(202,'Pune','411001'),
(203,'Nashik','422001'),
(204,'Bengaluru','560001');

INSERT INTO buyer VALUES
(301,'Karan Verma','9822000001','karan.verma@example.com'),
(302,'Ayesha Khan','9822000002','ayasha.khan@example.com'),
(303,'Mehul Shah','9822000003','mehul.shah@example.com'),
(304,'Isha Rao','9822000004','isha.rao@example.com'),
(305,'Vikram Singh','9822000005','vikram.singh@example.com');

INSERT INTO property VALUES
(1001,'Apartment',8500000,'Available',1,201,101),
(1002,'Villa',14500000,'Available',2,201,102),
(1003,'Plot',5200000,'Sold',3,202,103),
(1004,'Apartment',6800000,'Available',4,202,102),
(1005,'Villa',17500000,'Reserved',1,203,101),
(1006,'Apartment',9200000,'Available',2,204,103),
(1007,'Plot',4300000,'Available',3,203,101),
(1008,'Apartment',7600000,'Sold',4,201,102);

INSERT INTO buyerpropertyenquiry VALUES
(5001,301,1001,'2026-09-02','Interested in a site visit this weekend.','Contacted','2026-09-03'),
(5002,302,1002,'2026-09-03','Please share the floor plan and amenities.','Pending',NULL),
(5003,303,1004,'2026-09-04','Is the price negotiable?','Contacted','2026-09-05'),
(5004,304,1006,'2026-09-05','I would like a video tour.','Pending',NULL),
(5005,305,1007,'2026-09-06','Please confirm the exact plot dimensions.','Closed','2026-09-07'),
(5006,301,1004,'2026-09-08','Can I schedule a visit next week?','Contacted','2026-09-09'),
(5007,302,1001,'2026-09-09','Please send maintenance details.','Pending',NULL),
(5008,304,1002,'2026-09-10','Is home loan assistance available?','Contacted','2026-09-11');

```

12. SQL Queries Implemented

The project README specifies a query set covering listing retrieval, filtering, statistics, JOINS, enquiry tracking, grouping and view creation. The following queries implement those requirements.

12.1 View All Properties

```
SELECT *
FROM property
ORDER BY propertyid;
```

12.2 View Available Properties

```
SELECT *
FROM property
WHERE status = 'Available'
ORDER BY price;
```

12.3 Find Available Properties in Mumbai

```
SELECT p.propertyid, p.propertytype, p.price, l.city, l.pincode
FROM property p
JOIN location l ON p.locationid = l.locationid
WHERE p.status = 'Available'
AND l.city = 'Mumbai'
ORDER BY p.price;
```

12.4 Find Mumbai Properties Within a Price Range

```
SELECT p.propertyid, p.propertytype, p.price, l.city
FROM property p
JOIN location l ON p.locationid = l.locationid
WHERE p.status = 'Available'
AND l.city = 'Mumbai'
AND p.price BETWEEN 5000000 AND 15000000
ORDER BY p.price;
```

- With the sample data, this returns Property 1001 (₹85,00,000) and Property 1002 (₹1,45,00,000).

12.5 Total Number of Properties

```
SELECT COUNT(*) AS total_properties
FROM property;
```

12.6 Average Property Price

```
SELECT ROUND(AVG(price), 2) AS average_property_price
FROM property;
```

12.7 Property and Owner Details

```
SELECT p.propertyid, p.propertytype, p.price,
       o.ownername, o.phone, o.email
FROM property p
JOIN owner o ON p.ownerid = o.ownerid
ORDER BY p.propertyid;
```

12.8 Property and Agent Details

```
SELECT p.propertyid, p.propertytype, p.price,
       a.agentname, a.phone, a.email
FROM property p
```

```
JOIN agent a ON p.agentid = a.agentid
ORDER BY p.propertyid;
```

12.9 Track Buyer Enquiries

```
SELECT e.enquiryid, b.buyername, e.propertyid,
       e.enquirydate, e.message,
       e.followupstatus, e.followupdate
FROM buyerpropertyenquiry e
JOIN buyer b ON e.buyerid = b.buyerid
ORDER BY e.enquirydate;
```

12.10 Group Properties by City

```
SELECT l.city, COUNT(p.propertyid) AS property_count
FROM location l
LEFT JOIN property p ON l.locationid = p.locationid
GROUP BY l.city
ORDER BY property_count DESC, l.city;
```

12.11 Group Properties by Status

```
SELECT status, COUNT(*) AS property_count
FROM property
GROUP BY status
ORDER BY property_count DESC;
```

12.12 Group Properties by Agent

```
SELECT a.agentname, COUNT(p.propertyid) AS managed_properties
FROM agent a
LEFT JOIN property p ON a.agentid = p.agentid
GROUP BY a.agentid, a.agentname
ORDER BY managed_properties DESC, a.agentname;
```

12.13 Cities Having More Than One Property

```
SELECT l.city, COUNT(p.propertyid) AS property_count
FROM location l
JOIN property p ON l.locationid = p.locationid
GROUP BY l.city
HAVING COUNT(p.propertyid) > 1
ORDER BY property_count DESC, l.city;
```

13. AvailableListings View

The AvailableListings view is one of the main advanced concepts in the project. It joins property, owner, location and agent to provide one consolidated listing of properties whose status is Available.

```
CREATE VIEW AvailableListings AS
SELECT
    p.propertyid,
    p.propertytype,
    p.price,
    p.status,
    o.ownername,
    o.phone AS ownerphone,
    o.email AS owneremail,
    l.city,
    l.pincode,
    a.agentname,
    a.phone AS agentphone,
    a.email AS agentemail
FROM property p
JOIN owner o ON p.ownerid = o.ownerid
JOIN location l ON p.locationid = l.locationid
JOIN agent a ON p.agentid = a.agentid
WHERE p.status = 'Available';
```

13.1 Display the View

```
SELECT *
FROM AvailableListings
ORDER BY price;
```

13.2 Filter the View

```
SELECT propertyid, propertytype, price, city, agentname, agentphone
FROM AvailableListings
WHERE city = 'Pune'
    AND price BETWEEN 5000000 AND 10000000
ORDER BY price;
```

13.3 Why the View is Useful

- Agents get a single, ready-to-use listing result.
- The JOIN logic is defined once instead of repeated in every application query.
- Only currently available properties are exposed by the view.
- Changes to the underlying property status are reflected automatically when the view is queried.
- The view is suitable as a backend source for an agent dashboard or property listing screen.

14. Testing and Validation

Test	Validation	Expected Result
TC01	Insert valid owner	Record accepted
TC02	Insert property with valid owner/location/agent	Record accepted
TC03	Insert property with invalid ownerid	Foreign-key violation
TC04	Insert property with negative price	CHECK constraint violation
TC05	Run Mumbai + BETWEEN query	Only matching available listings returned
TC06	Create AvailableListings	View created successfully
TC07	Change Available property to Sold	It no longer appears in AvailableListings
TC08	Insert enquiry for valid buyer/property	Enquiry accepted
TC09	Insert enquiry with invalid buyerid	Foreign-key violation
TC10	Run HAVING city-count query	Only cities above the threshold returned

15. Integrity and Data Quality

15.1 Entity Integrity

- Every main table has a primary key.
- Primary keys prevent duplicate identifiers and provide stable record references.

15.2 Referential Integrity

- Property.ownerid must reference an existing owner.
- Property.locationid must reference an existing location.
- Property.agentid must reference an existing agent.
- BuyerPropertyEnquiry.buyerid must reference an existing buyer.
- BuyerPropertyEnquiry.propertyid must reference an existing property.

15.3 Domain Integrity

- Property price must be greater than zero.
- Property status is restricted to Available, Sold or Reserved.
- Follow-up status is restricted to Pending, Contacted or Closed.
- Important business fields are protected using NOT NULL constraints.

16. Project Outcomes

- Centralized management of property listings, owners, agents, locations, buyers and enquiries.
- Fast filtering of available properties by city and price range.
- Reduced redundancy through the separate Location relation.
- Reliable relationships through primary and foreign keys.
- Useful statistics using aggregate functions and GROUP BY/HAVING.
- Efficient enquiry follow-up through BuyerPropertyEnquiry.
- A reusable AvailableListings view for current property availability.
- A scalable database foundation for a future real-estate application.

17. Limitations and Future Enhancements

- Add property images, videos, floor plans and document attachments.
- Add site-visit appointment scheduling.
- Add buyer offers and negotiation history.
- Add authentication with Admin, Agent, Owner and Buyer roles.
- Add payment and booking modules.
- Add advanced filters such as bedrooms, area, furnishing and amenities.
- Add email/SMS/WhatsApp notifications for enquiries and follow-ups.
- Add audit history for changes to property price, status and ownership.
- Add analytics dashboards for inventory, enquiry conversion and agent performance.
- Add indexes and query optimization for production-scale datasets.

18. Conclusion

EstateHub successfully converts a real-world real-estate management problem into a structured PostgreSQL relational database. The project separates independent entities, establishes clear one-to-many relationships, and uses BuyerPropertyEnquiry to model the many-to-many interaction between buyers and properties.

The database demonstrates the core DBMS concepts expected from the case study: relational schema design, normalization, primary and foreign keys, SQL DDL and DML, JOINS, filtering with BETWEEN, aggregate functions, GROUP BY, HAVING and database views.

The AvailableListings view provides a particularly useful abstraction for agents because it presents current available property information together with owner, location and agent contact details. Overall, EstateHub provides a clean and extensible foundation for a real-world property listing and enquiry management application.

Appendix A – Complete SQL Reference

The following consolidated script represents the core PostgreSQL implementation documented in this report. Execute CREATE DATABASE separately, connect to estatehub_db, and then execute the table, data, query and view statements.

```
-- 1. Create database (run separately)
CREATE DATABASE estatehub_db;

-- 2. Connect to estatehub_db, then create tables
CREATE TABLE owner (
    ownerid INT PRIMARY KEY,
    ownername VARCHAR(100) NOT NULL,
    phone VARCHAR(15) NOT NULL,
    email VARCHAR(120) NOT NULL UNIQUE
);

CREATE TABLE agent (
    agentid INT PRIMARY KEY,
    agentname VARCHAR(100) NOT NULL,
    phone VARCHAR(15) NOT NULL,
    email VARCHAR(120) NOT NULL UNIQUE
);

CREATE TABLE location (
    locationid INT PRIMARY KEY,
    city VARCHAR(80) NOT NULL,
    pincode VARCHAR(10) NOT NULL
);

CREATE TABLE buyer (
    buyerid INT PRIMARY KEY,
    buyername VARCHAR(100) NOT NULL,
    phone VARCHAR(15) NOT NULL,
    email VARCHAR(120) NOT NULL UNIQUE
);

CREATE TABLE property (
    propertyid INT PRIMARY KEY,
    propertytype VARCHAR(40) NOT NULL,
    price NUMERIC(14,2) NOT NULL CHECK (price > 0),
    status VARCHAR(20) NOT NULL
    CHECK (status IN ('Available','Sold','Reserved')),
    ownerid INT NOT NULL REFERENCES owner(ownerid),
    locationid INT NOT NULL REFERENCES location(locationid),
    agentid INT NOT NULL REFERENCES agent(agentid)
);

CREATE TABLE buyerpropertyenquiry (
    enquiryid INT PRIMARY KEY,
    buyerid INT NOT NULL REFERENCES buyer(buyerid),
    propertyid INT NOT NULL REFERENCES property(propertyid),
    enquirydate DATE NOT NULL,
    message VARCHAR(500) NOT NULL,
    followupstatus VARCHAR(20) NOT NULL
    CHECK (followupstatus IN ('Pending','Contacted','Closed')),
    followupdate DATE
);

-- 3. View
CREATE VIEW AvailableListings AS
SELECT
    p.propertyid,
    p.propertytype,
    p.price,
    p.status,
    o.ownername,
    o.phone AS ownerphone,
    o.email AS owneremail,
    l.city,
    l.pincode,
    a.agentname,
```

```
    a.phone AS agentphone,  
    a.email AS agentemail  
FROM property p  
JOIN owner o ON p.ownerid = o.ownerid  
JOIN location l ON p.locationid = l.locationid  
JOIN agent a ON p.agentid = a.agentid  
WHERE p.status = 'Available';
```